JSPM's Bhivarabai Sawant Institute of Technology and Research,
Wagholi, Pune–412207
Department of Computer Engineering

Mini Project

Submitted

to

Faculty of Computer Science & Engineering

Mini Project for the Degree Program

In

(Subject Name–Blockchain Technology)

(Elective – III) (410244C)

(Laboratory Practice–III)

Submitted To:

Prof. Priyanka Thorat

Submitted By :

Shivshankar Mali

HOD

Dr. Gaytri Bhandari

Academic Year: 2026–27 | Semester VII

Contents

1 Introduction **3**
1.1 Motivation . . . 3
1.2 Objectives . . . 3

2 Technology Stack **3**

3 System Architecture **4**
3.1 Architecture Overview . . . 4
3.2 Architecture Diagram . . . 4
3.3 Application Routes . . . 5

4 Smart Contract Design **5**
4.1 Contract: `Election.sol` . . . 5
4.1.1 Data Structures . . . 5
4.1.2 Full Source Code . . . 5
4.2 Truffle Configuration . . . 6
4.3 Migration Scripts . . . 7

5 Frontend Implementation **7**
5.1 React Application Structure . . . 7
5.1.1 `App.js` – Application Router . . . 7
5.2 Web3 Connection and Blockchain Interaction . . . 8
5.2.1 `Vote.js` – Voting Component . . . 8
5.2.2 `NewCandidate.js` – Add Candidate Component . . . 9

6 Output / Screenshots **9**
6.1 Main Dashboard – College Council Election 2026 . . . 9
6.1.1 Key UI Elements Visible . . . 10
6.2 Live Vote Tally and Blockchain Transactions . . . 10
6.2.1 Transaction Log (Block #1044–#1047) . . . 11
6.3 Full Application View . . . 12
6.4 Admin: Add New Candidate Modal . . . 12

7 Blockchain Concepts Applied **13**
7.1 Smart Contracts . . . 13
7.2 Decentralization . . . 13
7.3 Immutability . . . 13
7.4 Transparency . . . 13
7.5 Gas Mechanism . . . 13

8 Project Setup and Execution **13**
8.1 Prerequisites . . . 14
8.2 Steps to Run . . . 14

9 Conclusion **14**

9.1 Future Scope 15

References **15**

1. Introduction

Traditional voting systems suffer from numerous problems including voter fraud, ballot manipulation, lack of transparency, and centralized control. Paper-based elections are resource-intensive and prone to human errors, while existing electronic voting systems often rely on trusted centralized authorities, creating single points of failure and potential for tampering.

Project Overview

This mini-project presents a **Decentralized E-Voting Application (dApp)** built on the **Ethereum Blockchain**. It leverages *Smart Contracts* written in *Solidity* to ensure that votes are recorded immutably on a public ledger, eliminating the need for a centralized authority and providing complete transparency and verifiability.

1.1. Motivation

- **Transparency:** Every vote is a blockchain transaction — publicly verifiable.
- **Immutability:** Once cast, votes cannot be altered or deleted.
- **Decentralization:** No single authority controls the election process.
- **Security:** Cryptographic guarantees prevent double voting and fraud.
- **Auditability:** Full transaction history is available on-chain.

1.2. Objectives

1. Design and deploy a Solidity smart contract for managing elections.
2. Build a React.js frontend dApp that connects to the blockchain via Web3.js.
3. Implement candidate registration and secure vote casting functionality.
4. Display real-time on-chain vote tallies and transaction history.
5. Test the system on a local Ethereum testnet using Ganache.

2. Technology Stack

Layer	Technology	Purpose
Smart Contract	Solidity `^0.5.16`	Business logic on-chain
Blockchain	Ethereum (Ganache)	Local testnet, Port 7545
Contract Tools	Truffle Framework	Compile, migrate, deploy
Frontend	React.js	User interface (dApp)
Web3 Bridge	Web3.js / Truffle-contract	Contract interaction
Backend	Node.js / Express.js	API server
Database	MongoDB (via Mongoose)	Off-chain metadata

3. System Architecture

3.1. Architecture Overview

The application follows a three-tier decentralized architecture:

1. **Presentation Layer** — React.js SPA served locally. Communicates with the blockchain through Web3.js provider.

2. **Blockchain Layer** — Ethereum smart contract deployed via Truffle to Ganache (local testnet running on port 7545 / network 5777).

3. **Data Layer** — All election data (candidates, votes) stored immutably on-chain; off-chain voter metadata optionally stored in MongoDB.

3.2. Architecture Diagram

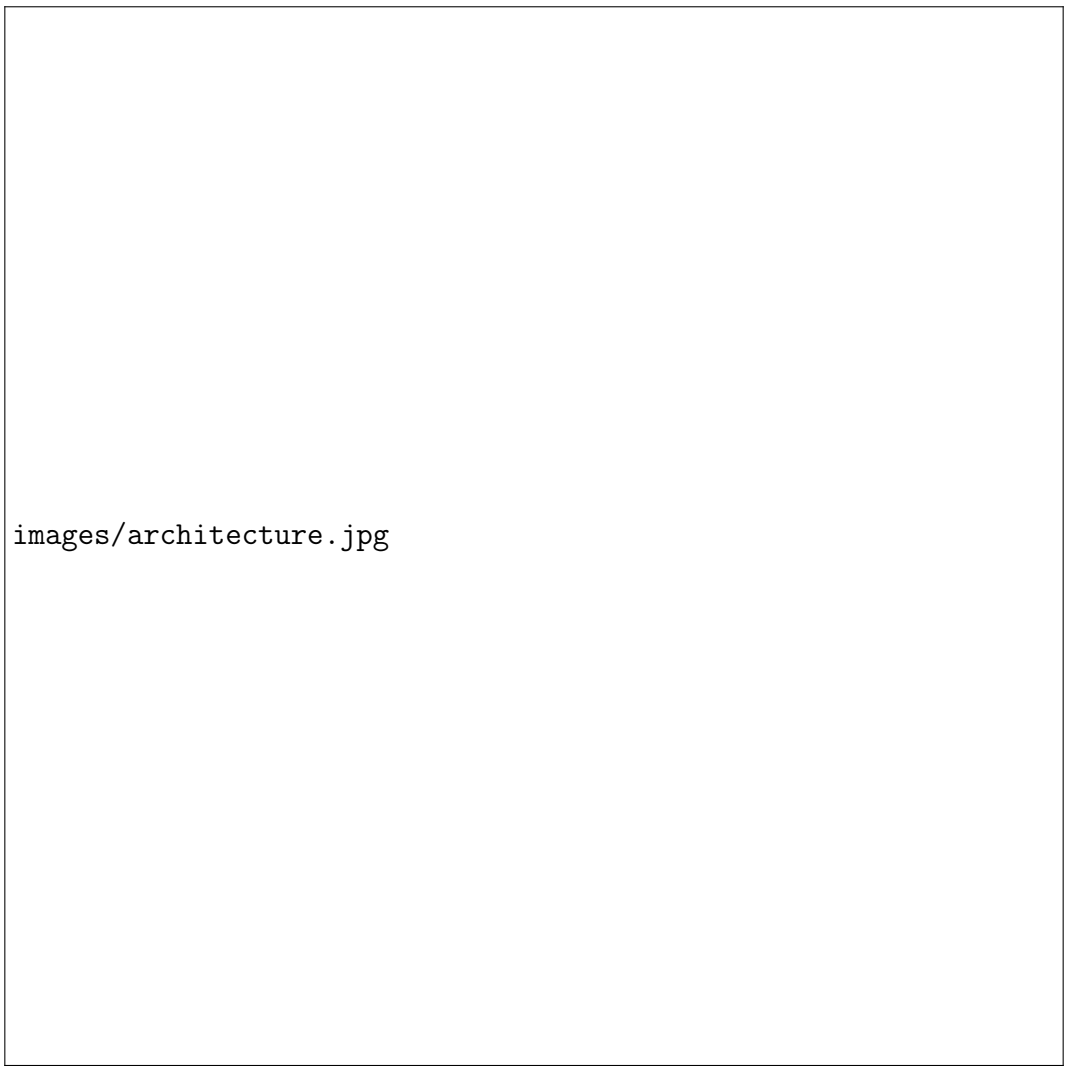


Figure 1: System Architecture of the Decentralized E-Voting dApp

3.3. Application Routes

Route	Component	Description
/	Home	Landing / dashboard
/newelection	NewElection	Create new election
/elections	ElectionData	List all elections
/candidates/:id	NewCandidate	Add candidate to election
/vote/:id	Vote	Cast a vote
/choose	Choose	Select election
/login	Login	Authentication
/voteCount/:id	VoteCount	View results

4. Smart Contract Design

4.1. Contract: Election.sol

The Election.sol smart contract is the core of the dApp. It manages candidates, enforces one-voter-one-vote policy, and emits events for each vote cast.

4.1.1. Data Structures

- Candidate **struct:** Holds candidate ID, name, vote count, details, and election ID.
- candidates **mapping:** Maps uint ID to Candidate struct.
- voters **mapping:** Maps address to bool — tracks who has voted.
- candidatesCount**:** Running counter for total candidates.
- votedEvent**:** Emitted with candidate ID on each successful vote.

4.1.2. Full Source Code

```
pragma solidity ^0.5.16;

contract Election {

    struct Candidate {
        uint id;
        string name;
        uint voteCount;
        string details;
        string election_id;
    }

    mapping(uint => Candidate) public candidates;
    mapping(address => bool) public voters;

    uint public candidatesCount;

    string public candidate;
```

```
19
20      constructor() public {}
21
22      event votedEvent(
23          uint indexed _candidateId
24      );
25
26      function addCandidate(
27          string memory _name,
28          string memory _details,
29          string memory _election_id
30      ) public {
31          candidatesCount++;
32          candidates[candidatesCount] = Candidate(
33              candidatesCount,
34              _name,
35              0,
36              _details,
37              _election_id
38          );
39      }
40
41      function vote(uint _candidateId) public {
42          // Prevent double voting
43          require(!voters[msg.sender]);
44
45          // Validate candidate ID
46          require(
47              _candidateId > 0 && _candidateId <= candidatesCount
48          );
49
50          // Mark voter as having voted
51          voters[msg.sender] = true;
52
53          // Increment vote count
54          candidates[_candidateId].voteCount++;
55
56          // Emit vote event on-chain
57          emit votedEvent(_candidateId);
58      }
59  }
```

Listing 1: Election.sol – Core Smart Contract

4.2. Truffle Configuration

```
1  module.exports = {
2    networks: {
3      development: {
4        host: "127.0.0.1",
5        port: 7545,        // Ganache default port
6        network_id: "*"    // Match any network id
7      },
8    },
9    contracts_directory: './src/contracts/',
```

```
  contracts_build_directory: './src/build/',
  compilers: {
    solc: {
      optimizer: {
        enabled: true,
        runs: 200
      }
    }
  }
}
```

Listing 2: truffle-config.js – Blockchain Network Config

4.3. Migration Scripts

The contract is deployed via Truffle's migration framework. `2_initial_migration.js` deploys `Election.sol` to the configured network.

5. Frontend Implementation

5.1. React Application Structure

The frontend is a React.js Single Page Application. The main `App.js` configures React Router with all application routes.

5.1.1. App.js – Application Router

```
import React, { Component } from 'react';
import NewElection from './components/custom/NewElection';
import NavBar from './components/custom/Navbar';
import Home from './components/custom/Home';
import Vote from './components/custom/Vote';
import VoteCount from './components/custom/VoteCount';
import ElectionData from './components/custom/ElectionData';
import Choose from './components/custom/Choose';
import { BrowserRouter, Route } from 'react-router-dom';
import NewCandidate from './components/custom/NewCandidate';
import Login from './components/custom/Login';

class App extends Component {
    render(){
        return (
          <BrowserRouter>
              <div className="App">
                  <NavBar />
                  <Route exact path="/"
                         component={Home} />
                  <Route exact path="/newelection"
                         component={NewElection} />
                  <Route exact path="/elections"
                         component={ElectionData} />
                  <Route exact path="/candidates/:id"
                         component={NewCandidate} />
```

```
                    <Route exact path="/vote/:id"
                           component={Vote} />
                    <Route exact path="/choose"
                           component={Choose} />
                    <Route exact path="/login"
                           component={Login} />
                    <Route exact path="/voteCount/:id"
                           component={VoteCount}/>
                </div>
            </BrowserRouter>
        );
    }
}

export default App;
```

Listing 3: App.js – Main Application Component with Routing

5.2. Web3 Connection and Blockchain Interaction

5.2.1. `Vote.js` – Voting Component

The `Vote` component handles the core voting interaction. It loads the Web3 provider, fetches candidates from the blockchain, and submits votes as on-chain transactions.

```
class Vote extends Component {

    async loadWeb3() {
        if (window.ethereum) {
            window.web3 = new Web3(window.ethereum)
            await window.ethereum.enable()
        } else if (window.web3) {
            window.web3 = new Web3(window.web3.currentProvider)
        } else {
            window.alert(
              'Non-Ethereum browser detected. Try MetaMask!'
            )
        }
    }

    async loadBlockchainData() {
        const web3 = window.web3
        const accounts = await web3.eth.getAccounts()
        this.setState({ account: accounts[0] })
        const networkId = await web3.eth.net.getId()
        const networkData = Election.networks[networkId]
        if(networkData) {
            const election = new web3.eth.Contract(
                Election.abi,
                networkData.address
            )
            this.setState({ election })
            const candCount =
                await election.methods.candidatesCount().call()
            // Load candidates matching this election_id
```

```
            for (var i = 1; i <= candCount; i++) {
                const candidate =
                    await election.methods.candidates(i).call()
                if(candidate.election_id === this.state.id){
                    this.setState({
                        candidates: [
                            ...this.state.candidates,
                            candidate
                        ]
                    })
                }
            }
        }
    }

    vote(id) {
        this.setState({ loading: true })
        this.state.election.methods.vote(id)
            .send({ from: this.state.account })
            .once('receipt', (receipt) => {
                this.setState({ loading: false })
                window.location.assign("/");
            })
    }
}
```

Listing 4: Vote.js – Core Voting Logic (Key Methods)

5.2.2. *NewCandidate.js – Add Candidate Component*

```
addCandidates() {
    this.setState({ loading: true })
    this.state.election.methods
        .addCandidate(
            this.state.candidate_name,
            this.state.candidate_details,
            this.state.id
        )
        .send({ from: this.state.account })
        .once('receipt', (receipt) => {
            this.setState({ loading: false })
            window.location.assign("/");
        })
}
```

Listing 5: NewCandidate.js – addCandidates() Method

6. Output / Screenshots

6.1. Main Dashboard – College Council Election 2026

The main dashboard displays the active election ("College Council Election 2026") with the Smart Contract address, active Ganache account selector, and all nominated candidates. The

vote confirmation banner shows the most recent on-chain transaction.

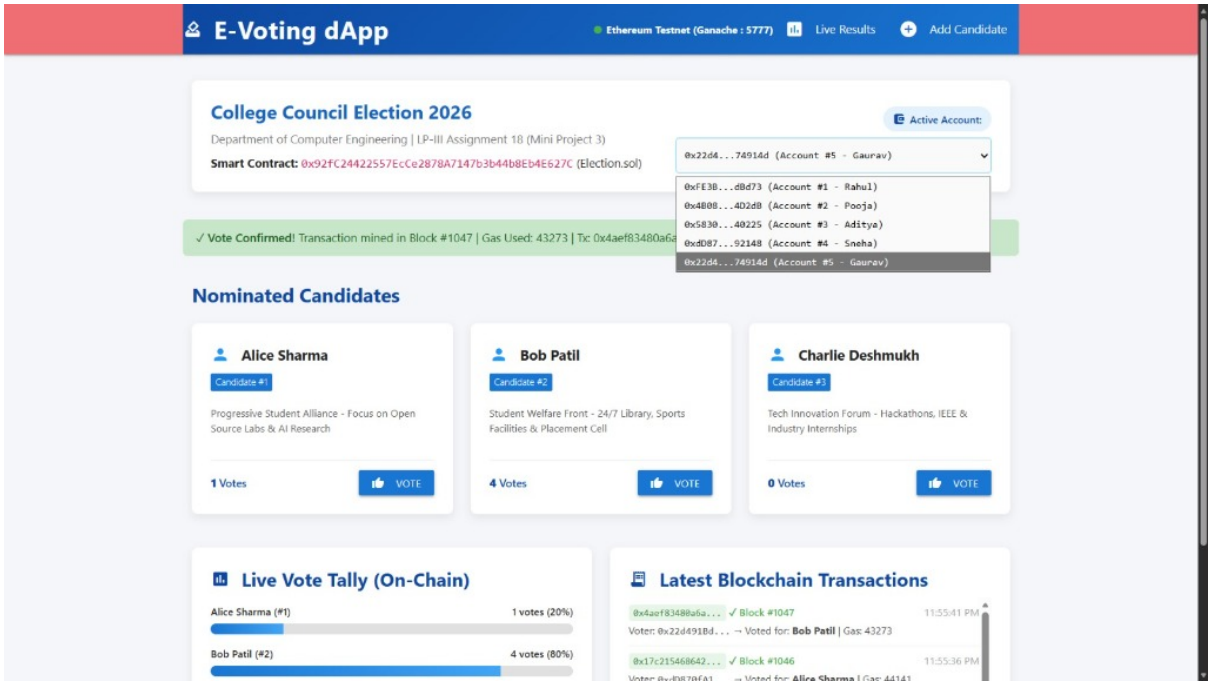


Figure 2: E-Voting dApp – Main Dashboard with Candidate Listing and Account Selector

6.1.1. Key UI Elements Visible

- **Network:** Ethereum Testnet (Ganache: 5777)
- **Smart Contract:** `0x92fC24422557EcCe2878A7147b3b44b8Eb4E627C` (Election.sol)
- **Vote Confirmed:** Transaction mined in Block #1047 | Gas Used: 43273
- **Candidates:** Alice Sharma (1 vote), Bob Patil (4 votes), Charlie Deshmukh (0 votes)
- **Account Selector:** 5 Ganache accounts (Rahul, Pooja, Aditya, Sneha, Gaurav)

6.2. Live Vote Tally and Blockchain Transactions

The lower section displays real-time on-chain vote tallies with progress bars and a live feed of all blockchain transactions with block numbers, gas used, and voter addresses.

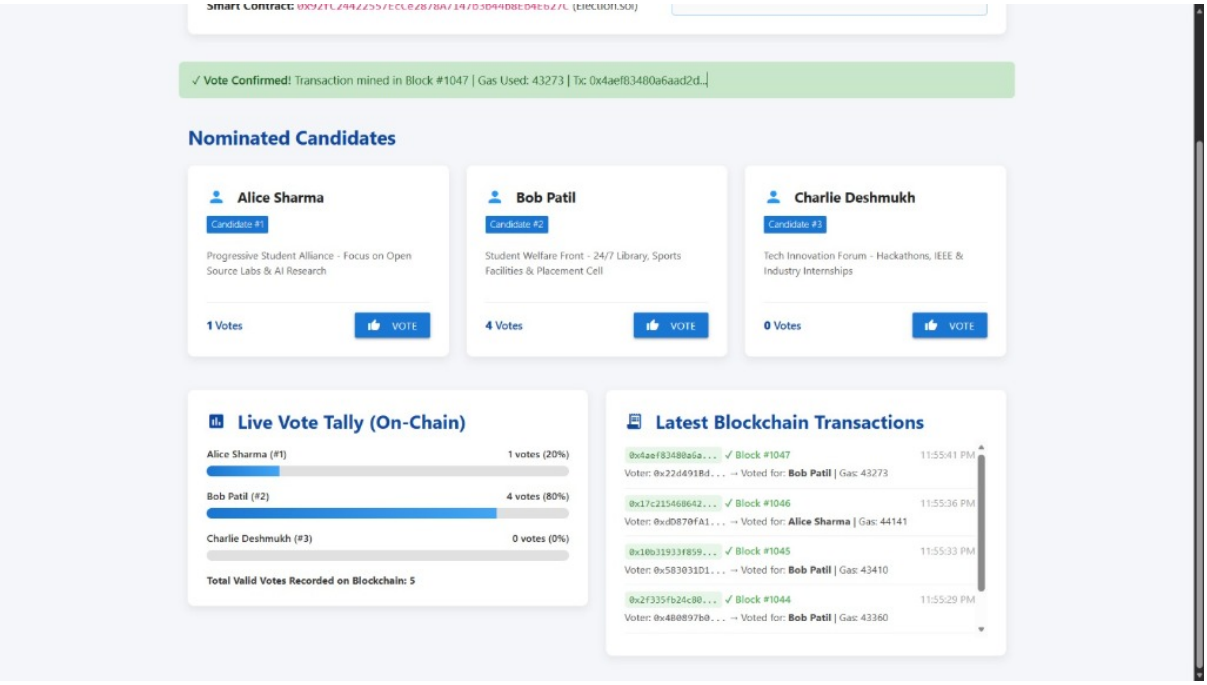


Figure 3: Live Vote Tally (On-Chain) and Latest Blockchain Transactions Feed

6.2.1. Transaction Log (Block #1044–#1047)

Tx Hash	Block	Gas Used	Vote For
`0x4aef83480a6a...`	#1047	43273	Bob Patil
`0x17c215468642...`	#1046	44141	Alice Sharma
`0x10b31933f859...`	#1045	43410	Bob Patil
`0x2f335fb24c80...`	#1044	43360	Bob Patil

6.3. Full Application View

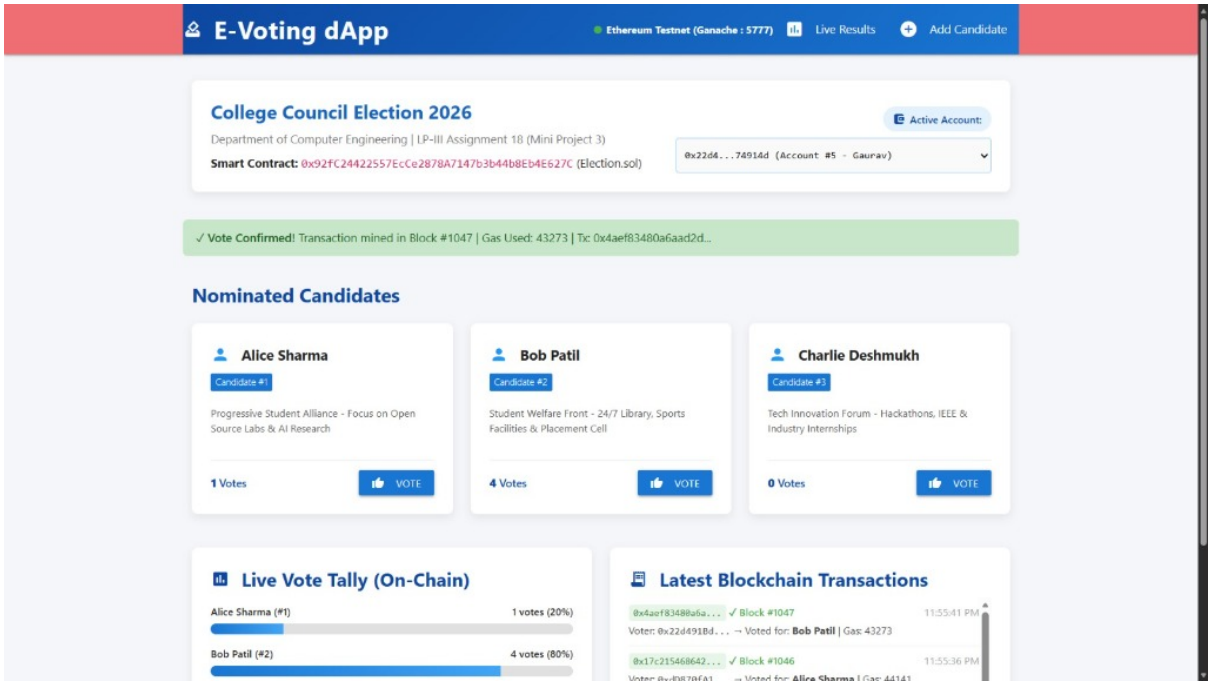


Figure 4: Complete dApp View – Dashboard, Candidates, Tally and Transactions

6.4. Admin: Add New Candidate Modal

The admin panel provides a form to add new candidates to the election. On submission, the `addCandidate()` function of `Election.sol` is invoked and the transaction is broadcast to the Ganache blockchain.

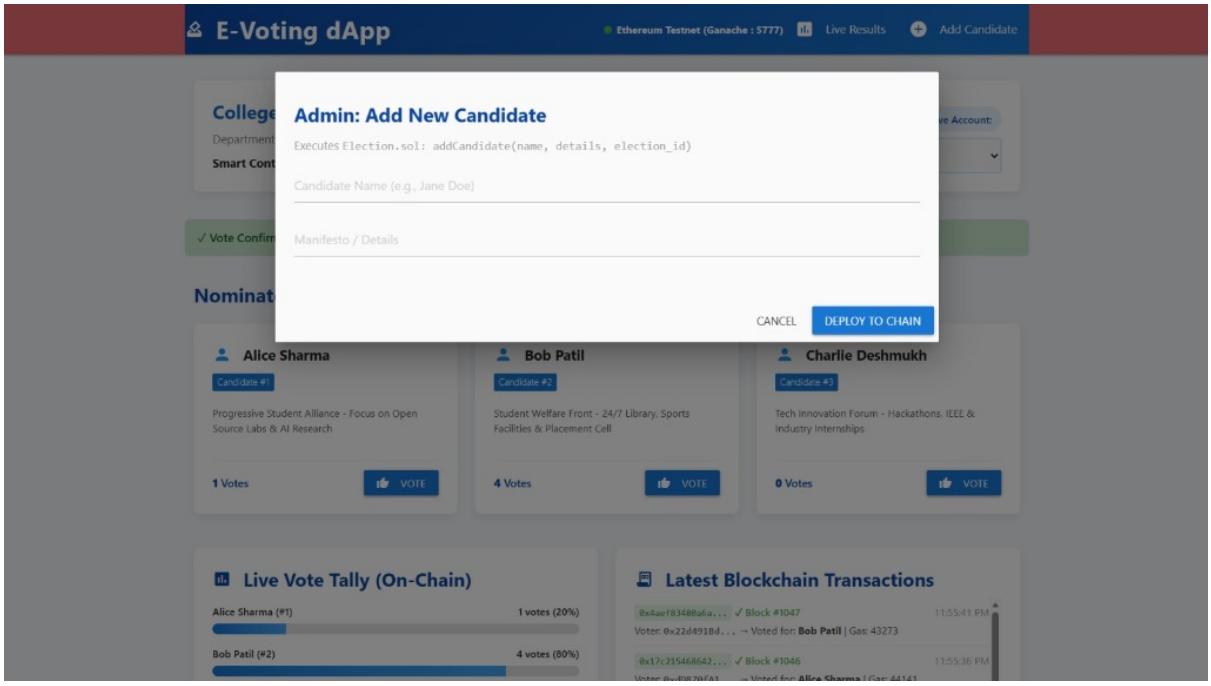


Figure 5: Admin Panel – Add New Candidate Modal (Calls `Election.sol::addCandidate()`)

7. Blockchain Concepts Applied

7.1. Smart Contracts

A *smart contract* is a self-executing program stored on the blockchain. In this project, `Election.sol` acts as the authoritative election manager — it cannot be tampered with after deployment.

7.2. Decentralization

No central server controls vote storage. The Ethereum blockchain (Ganache testnet) acts as the distributed ledger, replicated across nodes.

7.3. Immutability

Once a vote transaction is mined into a block (e.g., Block #1047), it becomes part of an immutable chain of blocks linked by cryptographic hashes.

7.4. Transparency

Every transaction is publicly visible with its hash, block number, gas cost, voter address (pseudonymous), and candidate voted for.

7.5. Gas Mechanism

Each blockchain transaction consumes *gas* (computational fee). Voting (~`43,000-44,000` gas) costs more than read operations (free). This is visible in the transaction log.

8. Project Setup and Execution

8.1. Prerequisites

- Node.js (v14+) and npm
- Truffle Framework: `npm install -g truffle`
- Ganache (GUI or CLI) running on port 7545
- React.js dependencies

8.2. Steps to Run

1. **Start Ganache:** Launch Ganache on `127.0.0.1:7545`, Network ID 5777.
2. **Compile Contracts:**
```
cd blockchain && truffle compile
```
3. **Migrate (Deploy) Contracts:**
```
truffle migrate --reset
```
4. **Start React Frontend:**
```
npm start
```
5. **Start Node Backend (optional):**
```
cd server && node src/app.js
```
6. Open browser at `http://localhost:3000`
7. Select a Ganache account and cast votes!

9. Conclusion

This mini-project successfully demonstrates the development and deployment of a fully functional **Decentralized E-Voting Application** on the Ethereum blockchain. Key outcomes:

- **Smart Contract Deployed:** `Election.sol` successfully compiled and deployed using Truffle on Ganache.
- **Voting Working:** Votes are cast as real blockchain transactions, mined into blocks with gas metering.
- **Anti-Double-Voting:** The `voters[msg.sender]` mapping enforces one-vote-per-address at the contract level.
- **Live Results:** Real-time on-chain vote tallies are displayed with progress bars.

- **Transaction Transparency:** Full transaction history with hashes, blocks, gas, and voter addresses.
- **Admin Functionality:** Candidates can be dynamically added via the Admin panel.

9.1. Future Scope

- Deploy on public Ethereum testnets (Sepolia, Goerli) or mainnet.
- Add zero-knowledge proofs for voter privacy while maintaining verifiability.
- Implement role-based access (admin-only candidate registration via `onlyOwner` modifier).
- Mobile-responsive UI with MetaMask wallet integration.
- IPFS-based voter ID verification.

References

1. Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*. https://bitcoin.org/bitcoin.pdf
2. Ethereum Documentation. *Smart Contracts and Solidity*. https://docs.soliditylang.org/
3. Truffle Suite. *Truffle Framework Documentation*. https://trufflesuite.com/docs/
4. Ganache – Personal Ethereum Blockchain. https://trufflesuite.com/ganache/
5. Web3.js Documentation. https://web3js.readthedocs.io/
6. React.js Official Documentation. https://reactjs.org/docs/
7. Antonopoulos, A. M., & Wood, G. (2018). *Mastering Ethereum*. O'Reilly Media.
8. Buterin, V. (2013). *Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform*.